

MediaPipe Gesture-AprilTag Id Sorting

1. Function Description

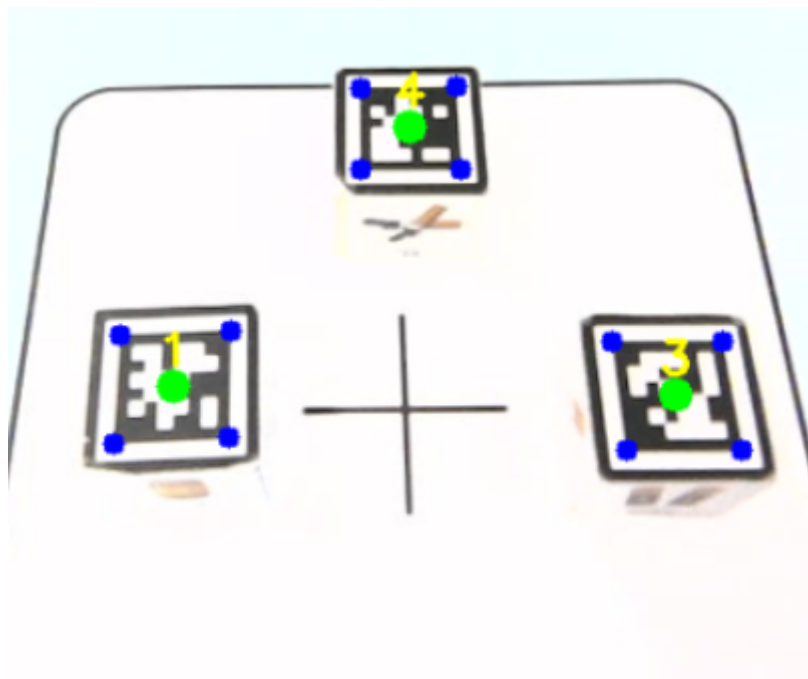
After the program runs, the camera captures images and recognizes gestures. The gestures are 1 to 4. By recognizing the gestures, the target AprilTag ID value is given. The robotic arm will then change its pose to search for the AprilTag with the target ID. If detected and recognized, it will grip and place it at the set position, then return to the recognition pose to continue recognition. If the target AprilTag is not detected, the robotic arm returns to the gesture recognition pose.

2. Startup and Operation

2.1 Startup Commands

Enter the following commands in the terminal:

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start mediapipe gesture recognition
ros2 run dofbot_sorting_3d mediapipe_gesture
# Start AprilTag recognition
ros2 run dofbot_sorting_3d apriltag_gesture_id
# Start robotic arm gripping
ros2 run dofbot_sorting_3d grasp
```



2.2 Operation

After the program starts, the robotic arm will initially present the gesture recognition pose. The recognizable gestures are 1 to 4, with corresponding gestures as shown below. Gesture recognition takes about 3 seconds to wait, then the robotic arm changes pose to the AprilTag detection and recognition pose. Press the spacebar to start recognition. If the target AprilTag is detected, the gripper will lower to grip it, then place it at the set position. If no AprilTag matching the target ID value is detected, the robotic arm returns to the initial gesture recognition pose.

3. Core Code Analysis

3.1 apriltag_gesture_id.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/apriltag_gesture_id.py
```

Import necessary libraries

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from message_filters import ApproximateTimeSynchronizer, Subscriber
from std_msgs.msg import Float32, Bool
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
import transforms3d as tfs
import tf_transformations as tf          # ROS2 uses tf_transformations
import threading
from dofbot_driver.vutils import draw_tags
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
import pyzbar.pyzbar as pyzbar
from std_msgs.msg import Float32, Bool, Int16
import time
import queue
import math
from Arm_Lib import Arm_Device
from dofbot_driver.compute_joint5 import *
```

Initialize program parameters, create publishers and subscribers

```
# AprilTag detection node class, inherited from rclpy.node.Node
# AprilTag Detection Node class, inherited from rclpy.node.Node
class AprilTagDetectNode(Node):
    def __init__(self):
        # Initialize ROS 2 node with name 'apriltag_detect'
        # Initialize ROS 2 node with name 'apriltag_detect'
        super().__init__('apriltag_detect')

        # Create an instance of the arm control device
        # Create an instance of the arm control device
        self.Arm = Arm_Device()

        # Initial joint angles configuration (unit: degrees)
        # Initial joint angles configuration (unit: degrees)
        self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]

        # Variable to store target angle for joint (Int16 type)
```

```

# Variable to store target angle for joint (Int16 type)
self.joint5 = Int16()

# Detection flag to control whether detection is active
# Detection flag to control whether detection is active
self.detect_flag = False

# Switch for height computation
# Switch for height computation
self.compute_height = True

# Index value for recording or locating target
# Index value for recording or locating target
self.index = None

# Create a client for kinematics service (type: Kinemarics, name:
'dofbot_kinemarics')
# Create a client for kinematics service (type: Kinemarics, name:
'dofbot_kinemarics')
self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')

# Publisher: publish target angle of joint to topic
# Publisher: publish target angle of joint to topic
self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)

# Publisher: publish target position info to "PosInfo" topic
# Publisher: publish target position info to "PosInfo" topic
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)

# Subscriber: listen to "grasp_done" topic, callback function is
GraspStatusCallback
# Subscriber: listen to "grasp_done" topic, callback function is
GraspStatusCallback
self.sub_grasp_done = self.create_subscription(Bool, 'grasp_done',
self.GraspStatusCallback, qos_profile=1)

# Subscriber: listen to "GestureId" topic, callback function is
GestureIDCallback
# Subscriber: listen to "GestureId" topic, callback function is
GestureIDCallback
self.sub_gestureID = self.create_subscription(Int16, "GestureId",
self.GestureIDCallback, 1)

# Publisher: publish reset gesture signal to topic
# Publisher: publish reset gesture signal to topic
self.pub_reset = self.create_publisher(Bool, "/reset_gesture",
qos_profile=10)

# Bridge to convert ROS RGB image messages to OpenCV format
# Bridge to convert ROS RGB image messages to OpenCV format
self.rgb_bridge = CvBridge()

# Flag to enable/disable position publishing
# Flag to enable/disable position publishing
self.pubPos_flag = False

# Record timestamp of last processing

```

```

# Record timestamp of last processing
self.pr_time = time.time()

# Create AprilTag detector with the following parameters:
# Create AprilTag detector with the following parameters:
self.at_detector = Detector(
    searchpath=['apriltags'],      # Search path
                                  # Search path for tag files
    families='tag36h11',          # Tag family used
                                  # Tag family used
    nthreads=8,                   # Number of threads
                                  # Number of threads
    quad_decimate=2.0,            # Quadrilateral decimation factor
                                  # Quadrilateral decimation factor
    quad_sigma=0.0,               # Gaussian blur sigma
                                  # Gaussian blur sigma
    refine_edges=1,               # Edge refinement switch
                                  # Edge refinement switch
    decode_sharpening=0.25,       # Decoding sharpening factor
                                  # Decoding sharpening factor
    debug=0                       # Debug mode switch
                                  # Debug mode switch
)

# List to store detected center X coordinates
# List to store detected center X coordinates
self.center_x_list = []

# List to store detected center Y coordinates
# List to store detected center Y coordinates
self.center_y_list = []

# Default distance parameter (unit: meters), used for coordinate conversion
# Default distance parameter (unit: meters), used for coordinate conversion
self.dist = 0.13

# Subscribe to RGB camera raw image topic "/image_raw", callback is
ImageCallback
# Subscribe to RGB camera raw image topic "/image_raw", callback is
ImageCallback
self.subscription = self.create_subscription(Image, '/image_raw',
self.ImageCallback, 1)

# Target AprilTag ID, initially 0 (no specific target)
# Target AprilTag ID, initially 0 (no specific target)
self.target_id = 0

# Detection switch to control whether to perform AprilTag detection
# Detection switch to control whether to perform AprilTag detection
self.detect = False

```

Image processing function ImageCallback

```

# Image callback function, processes RGB images from ROS topic and performs
AprilTag detection

```

```

# Image callback function, processes RGB images from ROS topic and performs
AprilTag detection
def ImageCallback(self, color_frame):
    # Convert ROS image message to OpenCV RGB format
    # Convert ROS image message to OpenCV RGB format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Copy RGB image for further processing and drawing detection results
    # Copy RGB image for further processing and drawing detection results
    result_image = np.copy(rgb_image)

    # Convert RGB image to grayscale and detect AprilTags
    # Convert RGB image to grayscale and detect AprilTags
    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)

    # Sort tags by tag_id in ascending order
    # Sort tags by tag_id in ascending order (may already be sorted)
    tags = sorted(tags, key=lambda tag: tag.tag_id)

    # Draw detected AprilTag borders and centers on the image
    # Draw detected AprilTag borders and centers on the image
    draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0,
255, 0))

    # Initialize center coordinate lists with length equal to number of detected
tags
    # Initialize center coordinate lists with length equal to number of detected
tags
    self.Center_x_list = list(range(len(tags)))
    self.Center_y_list = list(range(len(tags)))

    # Wait for key press
    # Wait for key press (10 ms delay)
    key = cv2.waitKey(10)

    # Enable position publishing when space key is pressed
    # Enable position publishing when space key is pressed
    if key == 32:
        self.pubPos_flag = True

    # If at least one AprilTag is detected
    # If at least one AprilTag is detected
    if len(tags) > 0:
        for i in range(len(tags)):
            # Check if current tag is the target tag
            # Check if current tag is the target tag
            if tags[i].tag_id == self.target_id:
                self.detect_flag = True
                self.pubPos_flag = True

            # Get the center coordinates of the tag
            # Get the center coordinates of the tag
            center_x, center_y = tags[i].center
            self.Center_x_list[i] = center_x
            self.Center_y_list[i] = center_y
            cx = center_x
            cy = center_y

```

```

        cz = self.dist

        # Convert pixel coordinates to robot coordinate system
        # Convert pixel coordinates to robot coordinate system
        (a, b) = (
            round(((cx - 320) / 4000), 5),
            round(((480 - cy) / 3000) * 0.8 + 0.13, 5)
        )
        print("a: ", a)
        print("b: ", b)

        # If position publishing is enabled, publish position info
        # If position publishing is enabled, publish position info
        if self.pubPos_flag == True:
            self.pubPos_flag = False
            pos = AprilTagInfo()
            pos.x = b
            pos.y = -a
            pos.z = 0.03
            pos.id = tags[i].tag_id
            self.pos_info_pub.publish(pos)

        # If target tag not detected and target_id is not 0, publish reset
signal
        # If target tag not detected and target_id is not 0, publish reset
signal
        if self.detect_flag == False and self.target_id != 0:
            reset = Bool()
            reset.data = True
            self.pub_reset.publish(reset)
            self.pubPos_flag = False

        # Convert RGB image to BGR format (required for OpenCV display)
        # Convert RGB image to BGR format (required for OpenCV display)
        result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)

        # Calculate and display FPS (frames per second)
        # Calculate and display FPS (frames per second)
        cur_time = time.time()
        fps = str(int(1 / (cur_time - self.pr_time)))
        self.pr_time = cur_time
        cv2.putText(result_image, fps, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)

        # Show the processed image in a window
        # Show the processed image in a window
        cv2.imshow("result_image", result_image)

        # Wait for key press (1ms delay)
        # Wait for key press (1 ms delay)
        key = cv2.waitKey(1)

```

3.2 mediapipe_gesture.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/mediapipe_gesture.py
```

Import necessary libraries

```
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image

from std_msgs.msg import Float32, Bool
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
import threading
from dofbot_sorting_3d.vutils import draw_tags
from dofbot_interface.srv import Kinematics
from dofbot_interface.msg import *
from std_msgs.msg import Float32, Bool, Int16
import time
from Arm_Lib import Arm_Device
from dofbot_sorting_3d.compute_joint5 import *
import os
```

Initialize program parameters, create publishers and subscribers,

```
def __init__(self):
    # Initialize the ROS2 node with the name 'apriltag_detect'
    # Initialize ROS2 node, node name is 'apriltag_detect'
    super().__init__('apriltag_detect')

    # Create an instance of the robotic arm device class
    # Create robotic arm device class instance
    self.Arm = Arm_Device()

    # Define initial joint angles for the robot arm
    # Define robotic arm initial joint angles
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]

    # Variable to store the value of joint (Int16 type)
    # Variable for storing joint value (Int16 type)
    self.joint5 = Int16()

    # Flag indicating whether a target has been detected
    # Flag indicating whether target is detected
    self.detect_flag = False

    # Flag indicating whether height calculation is needed
    # Flag indicating whether height calculation is needed
    self.compute_height = True
```

```

# Index of the detected AprilTag
# Detected AprilTag index
self.index = None

# Create a service client for kinematic calculations
# Create kinematic calculation client
self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')

# Publisher for publishing AprilTag position information
# Publisher for AprilTag position information
self.pos_info_pub = self.create_publisher(
    AprilTagInfo, "PosInfo", qos_profile=10)

# Subscriber for receiving grasp completion status
# Subscriber for grasp completion status
self.sub_grasp_done = self.create_subscription(
    Bool, 'grasp_done', self.GraspStatusCallback, qos_profile=1)

# Subscriber for receiving gesture ID commands
# Subscriber for gesture ID commands
self.sub_gestureID = self.create_subscription(
    Int16, "GestureId", self.GestureIDCallback, 1)

# Publisher for sending reset gesture command
# Publisher for sending reset gesture command
self.pub_reset = self.create_publisher(
    Bool, "/reset_gesture", qos_profile=10)

# Bridge for converting between OpenCV images and ROS2 Image messages
# Bridge for converting between OpenCV images and ROS2 Image messages
self.rgb_bridge = CvBridge()

# Flag indicating whether position publishing is active
# Flag indicating whether position publishing is active
self.pubPos_flag = False

# Previous time record for timing operations
# Previous time record for timing operations
self.pr_time = time.time()

# Initialize the AprilTag detector with specific parameters
# Initialize AprilTag detector with specific parameters
self.at_detector = Detector(searchpath=['apriltags'],
                             families='tag36h11',
                             nthreads=8,
                             quad_decimate=2.0,
                             quad_sigma=0.0,
                             refine_edges=1,
                             decode_sharpening=0.25,
                             debug=0)

# Distance parameter
# Distance parameter
self.dist = 0.13

# Subscriber for receiving raw camera images
# Subscriber for raw camera images
self.subscription = self.create_subscription(

```



```

        Image, '/image_raw', self.ImageCallback, 1)

# Target distance for detection (in meters)
# Detection target distance
self.target_dist = 0.14

# Detection flag
# Detection flag
self.detect = False

# Publisher for setting joint angle
# Publisher for setting joint angle
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)

```

Let's look at the main image callback function ImageCallback,

```

def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # Convert ROS2 Image message to OpenCV RGB image format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Create a copy of the RGB image for drawing results
    # Create RGB image copy for drawing results
    result_image = np.copy(rgb_image)

    # Detect AprilTags in the grayscale version of the image
    # Detect AprilTags in grayscale version of image
    # Convert RGB to Grayscale for detection
    # Convert RGB to grayscale for detection
    tags = self.at_detector.detect(cv2.cvtColor(
        rgb_image, cv2.COLOR_RGB2GRAY), False, None, 0.025)

    # Sort detected tags by their ID in ascending order
    # Sort detected tags by ID in ascending order
    # Tags may already be sorted, but explicit sorting ensures order
    # Tags may be sorted, explicit sorting ensures order
    tags = sorted(tags, key=lambda tag: tag.tag_id)

    # Draw bounding boxes and centers on the result image
    # Draw bounding boxes and centers on result image
    # Corners in red, center in green
    # Corners red, center green
    draw_tags(result_image, tags, corners_color=(
        0, 0, 255), center_color=(0, 255, 0))

    # Check for spacebar key press
    # Check for spacebar press
    # Pressing space enables position publishing
    # Press space to enable position publishing
    key = cv2.waitKey(10)
    if key == 32:
        self.pubPos_flag = True

    # Process each detected tag
    # Process each detected tag
    if len(tags) > 0:
        for i in range(len(tags)):

```

```

# Only process if position publishing is enabled
# Only process when position publishing is enabled
if self.pubPos_flag == True:
    # Get the center coordinates of the tag
    # Get tag center coordinates
    center_x, center_y = tags[i].center
    cx = center_x
    cy = center_y

    cz = self.dist

    # Calculate normalized coordinates (a, b)
    # Calculate normalized coordinates (a, b)
    # a: horizontal offset from image center, normalized
    # a: horizontal offset from image center, normalized
    # b: vertical offset from image center, normalized with scaling
factor
    # b: vertical offset from image center, with scaling factor
    (a, b) = (round(((cx - 320) / 4000), 5), round(((480 - cy) /
3000) * 0.8+0.13, 5))
    print("a: ", a)
    print("b: ", b)

    # Calculate edge vector for joint5 computation
    # Calculate edge vector for joint5 calculation
    # Difference between first two corners of the tag
    # Difference between first two corners of tag
    vx = int(tags[i].corners[0][0]) - \
        int(tags[i].corners[1][0])
    vy = int(tags[i].corners[0][1]) - \
        int(tags[i].corners[1][1])

orientation
    # Compute the required angle for joint 5 based on edge

    # Compute required angle for joint 5 based on edge orientation
    target_joint5 = compute_joint5(vx, vy)
    print("target_joint5: ", target_joint5)

reached
    # If position publishing is enabled and target distance is

    # If position publishing enabled and target distance reached
    if self.pubPos_flag == True and b <= self.target_dist:
        # Disable further position publishing
        # Disable further position publishing
        self.pubPos_flag = False

        # Create and publish joint5 command
        # Create and publish joint5 command
        joint5 = Int16()
        joint5.data = int(target_joint5)
        self.pub_joint5.publish(joint5)

        # Create and publish AprilTag position information
        # Create and publish AprilTag position information
        pos = AprilTagInfo()
        pos.x = float(b)
        pos.y = float(-a)
        pos.z = 0.03

```

```

        pos.id = tags[i].tag_id # Tag identifier
        self.pos_info_pub.publish(pos)

# If no valid tag was found at target distance
# If no valid tag found at target distance
if self.detect_flag == False and self.pubPos_flag == True:
    # Send reset command to gesture system
    # Send reset command to gesture system
    reset = Bool()
    reset.data = True
    self.pub_reset.publish(reset)

    # Reset position publishing flag
    # Reset position publishing flag
    self.pubPos_flag = False

    # Reset target distance to default value
    # Reset target distance to default value
    self.target_dist = 0.14
    print("Do not find the target dist tag.")

# Convert result image back to BGR format for display
# Convert result image back to BGR format for display
result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)

# Calculate and display FPS (frames per second)
# Calculate and display FPS
cur_time = time.time()
fps = str(int(1/(cur_time - self.pr_time)))
self.pr_time = cur_time
cv2.putText(result_image, fps, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Display the result image in a window
# Display result image in window
cv2.imshow("result_image", result_image)
key = cv2.waitKey(1)

```

Gesture detection function Gesture_Detect_threading

```

# Gesture detection thread function, determine number of raised fingers and
publish gesture ID after stabilization
# Gesture detection thread function: determine number of raised fingers and
publish gesture ID after stabilization
def Gesture_Detect_threading(self, lmList, bbox):
    # Get list of finger states (1 = extended, 0 = bent)
    # Get finger state list (1 = extended, 0 = bent)
    fingers = self.hand_detector.fingersUp(lmList)
    print("sum of fingers: ", sum(fingers))
    print(self.pub_gesture)

    # If current finger count equals last sum, increment counter
    # If current finger count equals last sum, increment counter
    if sum(fingers) == self.last_sum:
        print("-----")
        self.cnt = self.cnt + 1
        print("cnt: ", self.cnt)

```

```

        # When 30 consecutive frames have same count and publishing is allowed,
confirm gesture and publish
        # When 30 consecutive frames same and publishing allowed, confirm
gesture and publish
        if self.cnt == 30 and self.pub_gesture == True:
            print("sum of fingers: ", self.last_sum)
            self.pub_gesture = False # Prevent repeated publishing
                                     # Prevent repeated publishing

            sum_gesture = Int16()
            sum_gesture.data = self.last_sum
            self.pub_GestureId.publish(sum_gesture) # Publish gesture ID
                                                    # Publish gesture ID

            # Move arm to grasping pose
            # Move arm to grasping pose
            self.Arm.Arm_serial_servo_write6_array(self.grasp_joints, 2000)
        else:
            # Finger state changed, reset counter
            # Finger state changed, reset counter
            self.cnt = 0

        # Update last sum of fingers
        # Update last finger sum
        self.last_sum = sum(fingers)

```

3.3 grasp.py

The previous tutorial has explained this, so I won't repeat it here.